

Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Octree Voxelization Algorithm

Samuelson Dharmawan Tanurahrja – 13524001
Reinhard Alfonzo Hutabarat – 13524056

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung, Jalan Ganesha 10 Bandung
Email: 13524001@std.stei.itb.ac.id, 13524056@std.stei.itb.ac.id

CONTENTS

I	Deskripsi Permasalahan	2
II	Pendekatan Solusi	2
II-A	Algoritma Divide and Conquer	2
II-A1	Divide	2
II-A2	Conquer	2
II-A3	Combine	2
II-B	Konversi Model menggunakan Algoritma Divide and Conquer	2
II-B1	Fase Divide	2
II-B2	Fase Conquer	3
II-B3	Fase Combine	3
III	Source Code Program	3
III-A	Vector3.hpp	3
III-B	Triangle.hpp	4
III-C	AABB.hpp dan AABB.cpp	4
III-D	Octree.hpp dan Octree.cpp	4
III-E	main.cpp	5
IV	Hasil Pengujian	7
V	Pranala Repository	9

I. DESKRIPSI PERMASALAHAN

Permasalahan utama dalam tugas ini adalah bagaimana mengonversi model 3D berbasis *polygon mesh* menjadi representasi volumetrik berupa voxel (kubus 3D) secara mangkus dan efisien. Melakukan pengecekan irisan secara konvensional (*brute-force*) pada seluruh ruang 3D akan memakan beban komputasi yang sangat masif seiring bertambahnya resolusi. Oleh karena itu, permasalahan ini diselesaikan menggunakan strategi algoritma Divide and Conquer melalui struktur data spasial Octree. Algoritma akan membagi ruang pembungkus objek menjadi delapan sub-ruang secara rekursif, dan secara cerdas memangkas waktu komputasi dengan hanya melanjutkan pembagian pada sub-ruang yang terbukti beririsan dengan permukaan objek hingga mencapai batas kedalaman (*max depth*) tertentu.

II. PENDEKATAN SOLUSI

A. Algoritma Divide and Conquer

Algoritma Divide and Conquer adalah teknik pemecahan masalah yang digunakan untuk menyelesaikan masalah dengan membagi masalah utama menjadi submasalah, menyelesaikannya satu per satu, lalu menggabungkannya kembali untuk menemukan solusi atas masalah awal. Algoritma ini berguna ketika kita membagi suatu masalah menjadi submasalah yang saling independen. Berikut definisi dari 3 komponen utama dalam algoritma ini:

1) *Divide*: Memecah persoalan menjadi beberapa upa-persoalan yang memiliki kemiripan dengan persoalan semula namun berukuran lebih kecil. Setiap upa-persoalan harus mewakili bagian dari masalah keseluruhan. Tujuannya adalah membagi masalah tersebut hingga tidak mungkin lagi untuk membaginya lebih lanjut.

2) *Conquer*: Menyelesaikan setiap upa-persoalan kecil secara terpisah. Jika suatu upa-persoalan cukup kecil atau *base case*, kita menyelesaikannya secara langsung tanpa perlu melakukan rekursi lebih lanjut. Tujuannya adalah untuk menemukan solusi bagi submasalah-submasalah tersebut secara terpisah

3) *Combine*: Gabungkan submasalah-submasalah tersebut untuk mendapatkan solusi akhir dari keseluruhan masalah. Setelah submasalah-submasalah yang lebih kecil terpecahkan, kita menggabungkan solusinya secara rekursif untuk mendapatkan solusi dari masalah yang lebih besar. Tujuannya adalah merumuskan solusi untuk masalah awal dengan menggabungkan hasil dari submasalah-submasalah tersebut.

B. Konversi Model menggunakan Algoritma Divide and Conquer

Proses *voxelization* bertujuan untuk mengubah model 3D yang awalnya direpresentasikan sebagai kumpulan poligon (segitiga) kontinu menjadi representasi diskrit berupa kumpulan kubus bervolume (*voxels*). Untuk menangani jutaan titik spasial secara efisien, program ini menggunakan struktur data *Octree* yang mengimplementasikan paradigma *Divide and Conquer*.

Sebelum algoritma utama dijalankan, tahap inisialisasi dilakukan dengan membaca *file* masukan (berformat `.obj`) untuk mengekstrak seluruh titik (*vertex*) dan wajah (*face* segitiga). Setelah itu, program menghitung *Bounding Box* utama berupa kubus yang mencakup keseluruhan model 3D tersebut di dalamnya. Kubus induk inilah yang akan diproses ke dalam algoritma *Divide and Conquer* dengan langkah-langkah sebagai berikut:

1) *Fase Divide*: Pada fase ini, ruang 3D (direpresentasikan oleh objek *Axis-Aligned Bounding Box* atau AABB) dibagi menjadi upa-persoalan yang lebih kecil. Sebuah kubus AABB dipotong tepat di tengah-tengah sumbu X, Y, dan Z sehingga menghasilkan tepat 8 kubus anak (*octants*) yang berukuran setengah dari kubus induknya. Titik pusat (*center*) dari masing-masing anak dihitung dengan menggeser titik pusat induk ke delapan arah sudut yang berbeda berdasarkan nilai vektor *half-extents* yang baru. Pemotongan ini dilakukan secara rekursif.

2) *Fase Conquer*: Setiap upa-persoalan (kubus anak) dievaluasi secara independen. Karena tujuan utamanya adalah memfilter ruang kosong, fase *Conquer* memiliki dua kondisi basis (*base case*) untuk menghentikan rekursi:

- 1) *Basis Pruning*: Setiap kubus anak akan diuji apakah ia bersentuhan (*intersect*) dengan salah satu segitiga dari model 3D. Pengujian ini menggunakan *Separating Axis Theorem* (SAT) dengan mengevaluasi proyeksi pada 13 sumbu kritis (3 sumbu normal AABB, 1 sumbu normal segitiga, dan 9 sumbu *cross product*). Jika terbukti kubus tersebut tidak menyentuh segitiga manapun, maka kubus tersebut dianggap sebagai ruang kosong. Rekursi dihentikan seketika dan cabang *tree* tersebut dipangkas (*pruning*) untuk menghemat beban komputasi.
- 2) *Basis Pembentukan Voxel*: Jika sebuah kubus terbukti menyentuh segitiga model dan rekursi telah mencapai batas kedalaman maksimal (`maxDepth`) yang ditentukan oleh pengguna, maka kubus tersebut tidak lagi dibagi. Kubus ini diresmikan sebagai satu *voxel solid* penyusun model 3D akhir.
- 3) *Fase Combine*: fase *Combine* pada pembentukan struktur data *Octree* bersifat agregatif, dimana setelah seluruh pemanggilan rekursif selesai dieksekusi, program melakukan penelusuran kembali ke seluruh simpul daun (*leaf nodes*) yang bertahan (tidak terkena *pruning* dan mencapai `maxDepth`). Kubus-kubus *voxel* dari berbagai cabang *Octree* ini kemudian dikumpulkan dan digabungkan ke dalam satu struktur data *array* linier (`std::vector<AABB>`). Kumpulan *voxel* inilah yang menjadi solusi akhir dari keseluruhan persoalan, yang siap untuk divisualisasikan oleh *3D Viewer* serta ditulis kembali menjadi *file .obj* baru.

III. SOURCE CODE PROGRAM

Pada bagian ini, akan dijelaskan secara singkat komponen-komponen inti dari kode sumber yang menyusun program *voxelization* ini, khususnya pada modul matematika dasar dan geometri spasial. Potongan kode yang ditampilkan merupakan representasi dari logika utama pada masing-masing *file*.

A. *Vector3.hpp*

Tujuan: *File* ini merupakan fondasi matematika dari seluruh program. `Vector3` digunakan untuk merepresentasikan banyak hal di dalam ruang 3D, mulai dari posisi titik (*vertex*), jarak (*extents*), hingga arah vektor (*normal* dan cahaya).

Komponen Utama:

```
1 struct Vector3 {
2     float x, y, z;
3     // ... (Konstruktor dan Operator aritmatika) ...
4
5     Vector3 cross(const Vector3 &v) const {
6         return Vector3(
7             y * v.z - z * v.y,
8             z * v.x - x * v.z,
9             x * v.y - y * v.x);
10    }
11
12    float dot(const Vector3 &v) const {
13        return x * v.x + y * v.y + z * v.z;
14    }
15};
```

Penjelasan: Bagian terpenting dari struktur data ini adalah operasi *Cross Product* (`cross`) dan *Dot Product* (`dot`). Keduanya sangat krusial dan dipanggil ribuan kali setiap detiknya untuk menghitung sumbu proyeksi pada algoritma *Separating Axis Theorem* (SAT) serta untuk perhitungan pencahayaan (*Flat Shading*) pada *3D Viewer*.

B. Triangle.hpp

Tujuan: Struktur data sederhana untuk merepresentasikan satu poligon wajah (*face*) dari model .obj mentah yang selalu terdiri dari tiga titik sudut (*triangle mesh*).

Komponen Utama:

```
1 struct Triangle {
2     Vector3 v0, v1, v2;
3     Triangle(const Vector3& a, const Vector3& b, const Vector3& c) : v0(a), v1(b),
4         v2(c) {}
5 };
```

Penjelasan: Setiap segitiga menyimpan tiga buah `Vector3` yang mewakili posisinya di dunia 3D. Objek dari *struct* inilah yang nantinya akan diuji persinggungannya dengan kubus *Octree*.

C. AABB.hpp dan AABB.cpp

Tujuan: Mendefinisikan *Axis-Aligned Bounding Box* (AABB) yang merepresentasikan sebuah kubus yang rusuknya selalu sejajar dengan sumbu utama pada dimensi tersebut. *File* ini juga menyimpan implementasi algoritma deteksi perpotongan 2 objek menggunakan *Separating Axis Theorem* (SAT).

Komponen Utama:

```
1 bool AABB::intersects(const Triangle &tri) const {
2     // ... (Mencari vektor rusuk segitiga) ...
3
4     // Uji 1: 3 Sumbu Normal AABB (Sumbu X, Y, Z)
5     if (minX > halfExtents.x || maxX < -halfExtents.x) return false;
6
7     // Uji 2: 1 Sumbu Normal Segitiga
8     Vector3 triangleNormal = e0.cross(e1);
9     if (std::abs(planeDistance) > r) return false;
10
11     // Uji 3: 9 Sumbu Silang (Cross Product)
12     Vector3 crossAxes[9] = { /* ... 9 kombinasi cross product ... */ };
13     for (int i = 0; i < 9; ++i) {
14         // Jika pada satu sumbu saja bayangan tidak tumpang tindih,
15         // maka dipastikan kotak dan segitiga TIDAK BERSENTUHAN.
16         if (minP > rad || maxP < -rad) return false;
17     }
18
19     return true; // Bersentuhan jika lolos semua 13 uji sumbu
20 }
```

Penjelasan: Fungsi `intersects` menjadi inti dari optimasi program. Fungsi ini memproyeksikan kubus AABB dan Segitiga ke dalam 13 sumbu imajiner. Jika terdapat setidaknya satu sumbu di mana proyeksi keduanya tidak tumpang tindih, maka terbukti secara matematis bahwa kubus tersebut kosong (tidak berisi potongan model 3D).

D. Octree.hpp dan Octree.cpp

Tujuan: Menangani proses pemecahan ruang 3D secara rekursif. *File* ini merupakan implementasi langsung dari paradigma algoritma *Divide and Conquer*.

Komponen Utama:

```
1 void Octree::subdivideRecursive(OctreeNode* node, const std::vector<Triangle>&
2     triangles, int currentDepth) {
3     // ... (Cari segitiga yang bersentuhan dengan node ini) ...
4
5     // BASIS 1: Pruning (Ruang Kosong)
6     if (intersectingTriangles.empty()) return;
```

```

7 // BASIS 2: Voxel Terbentuk (Kedalaman Maksimal)
8 if (currentDepth == maxDepth) {
9     totalVoxels++;
10    return;
11 }
12
13 // DIVIDE: Membagi kubus menjadi 8 oktan
14 Vector3 newHE = node->boundingBox.halfExtents * 0.5f;
15 for (int i = 0; i < 8; ++i) {
16     // ... (Hitung titik pusat/center untuk anak ke-i) ...
17     AABB childBox(childCenter, newHE);
18     node->children[i] = std::make_unique<OctreeNode>(childBox);
19
20     // CONQUER: Rekursi ke anak
21     subdivideRecursive(node->children[i].get(), intersectingTriangles,
22                        currentDepth + 1);
23 }

```

Penjelasan: Potongan kode di atas menunjukkan alur *Divide and Conquer*. Ruang dibagi menjadi 8 secara rekursif. Basis 1 (*Pruning*) akan membuang komputasi pada ruang yang tidak bersentuhan dengan segitiga model, menjadikannya sangat efisien. Basis 2 akan menghentikan rekursi jika batas ukuran resolusi (kedalaman) kubus *voxel* telah tercapai.

E. main.cpp

Tujuan: Menjadi *entry point* program yang menjalankan seluruh alur kerja sistem, mulai dari pembacaan *file* mentah 3D, eksekusi algoritma *Divide and Conquer*, hingga penulisan hasil dan pemanggilan antarmuka visualisasi.

Komponen Utama:

```

1 // --- 1. ALUR UTAMA (main.cpp) ---
2 int main (int argc, char* argv[]) {
3     // PROSES 1: Parsing file input menggunakan ObjParser
4     ObjParser parser;
5     parser.parse(inputPath, triangles, minBound, maxBound);
6
7     // MENGHITUNG BOUNDING BOX UTAMA (ROOT)
8     Vector3 center = Vector3(
9         (minBound.x + maxBound.x) / 2.0f,
10        (minBound.y + maxBound.y) / 2.0f,
11        (minBound.z + maxBound.z) / 2.0f
12    );
13    Vector3 halfExtents = Vector3(
14        (maxBound.x - minBound.x) / 2.0f,
15        (maxBound.y - minBound.y) / 2.0f,
16        (maxBound.z - minBound.z) / 2.0f
17    );
18
19    // Memaksa Bounding Box menjadi kubus sempurna (Cubic)
20    float maxHE = std::max({halfExtents.x, halfExtents.y, halfExtents.z});
21    Vector3 cubicHalfExtents(maxHE, maxHE, maxHE);
22    AABB rootBox(center, cubicHalfExtents);
23
24    // PROSES 2 & 3: Divide & Conquer dan Ekstraksi Voxel
25    Octree octree(rootBox, maxDepth);
26    octree.build(triangles);
27    vector<AABB> finalVoxels = octree.getFinalVoxels();
28
29    // PROSES 4: Menulis output menggunakan ObjWriter
30    ObjWriter writer;
31    writer.writeVoxels(outputPath, finalVoxels);
32
33    // PROSES 5: Membuka Interactive 3D Viewer

```

```

34     Viewer viewer(finalVoxels);
35     viewer.run();
36
37     return 0;
38
39 }
40
41 // --- 2. MODUL PARSER (ObjParser.cpp) ---
42 bool ObjParser::parse(...) {
43     // Membaca file baris demi baris, mengekstrak vertex (v)
44     // dan merangkainya menjadi sekumpulan segitiga (f).
45     // Modul ini juga mencari titik ekstrem untuk membentuk rootBox Octree.
46 }
47
48 // --- 3. MODUL WRITER (ObjWriter.cpp) ---
49 bool ObjWriter::writeVoxels(...) {
50     // Mengonversi setiap AABB (voxel) menjadi 8 titik sudut (v)
51     // dan 12 sisi segitiga (f) agar menjadi format kubus .obj yang utuh.
52 }

```

Penjelasan: Potongan kode di atas menunjukkan *pipeline* program secara terstruktur dari awal hingga akhir. Pertama, program mendelegasikan tugas prapemrosesan kepada `ObjParser` untuk membaca *file* `.obj` baris demi baris. Modul ini menerjemahkan teks menjadi representasi simpul matematis, sekaligus mencari titik ekstrem (`minBound` dan `maxBound`) dari keseluruhan model.

Titik ekstrem tersebut kemudian digunakan untuk mencari titik tengah (`center` dan jarak ke ujung (`halfExtents`) dari model. Untuk memastikan bahwa pembagian ruang oleh *Octree* presisi dan simetris, nilai `halfExtents` terbesar (`maxHE`) diambil dan diaplikasikan ke ketiga sumbu (X, Y, Z) agar menghasilkan *Bounding Box* utama (`rootBox`) yang berbentuk kubus sempurna.

Selanjutnya, `rootBox` tersebut diserahkan ke `Octree` untuk dipotong-potong secara spasial menggunakan algoritma *Divide and Conquer*. Setelah proses rekursi selesai, data kumpulan *voxel* yang terbukti beririsan ditarik dan dikirim ke `ObjWriter`. Di dalam tahapan pas-capemrosesan ini, kotak-kotak abstrak tersebut diekspansi kembali; satu *voxel* direkonstruksi menjadi 8 titik sudut dan 12 jaring segitiga agar dapat disimpan sebagai *file* 3D balok yang solid. Terakhir, data *voxel* tersebut diteruskan ke kelas `Viewer` untuk di-*render* ke layar secara visual.

IV. HASIL PENGUJIAN

Berikut adalah pengujian program menggunakan berbagai konfigurasi papan masukan beserta hasil visualisasinya.

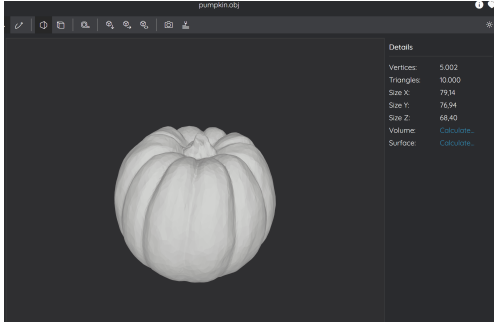
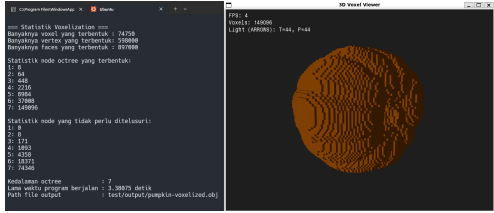
Test Case 1	
Input	Output
 $maxDepth = 7$	

Fig. 1. Hasil Uji Coba Test Case 1

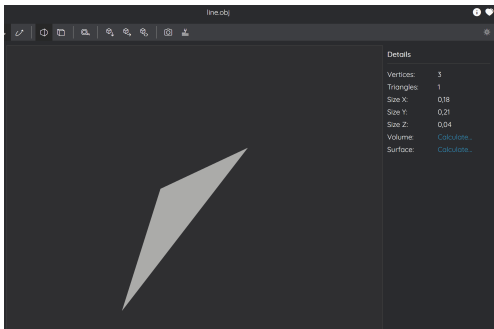
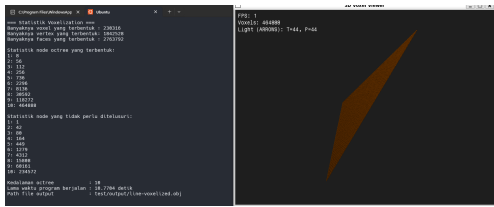
Test Case 2	
Input	Output
 $maxDepth = 10$	

Fig. 2. Hasil Uji Coba Test Case 2

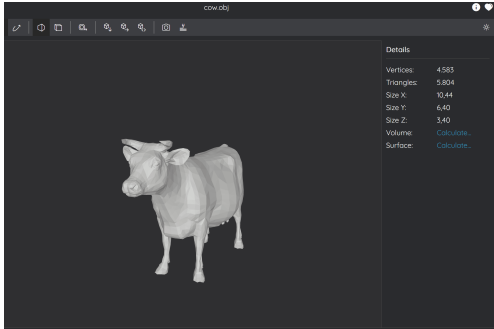
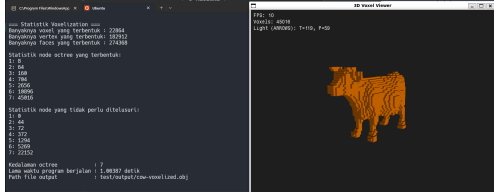
Test Case 3	
Input	Output
 $maxDepth = 7$	

Fig. 3. Hasil Uji Coba Test Case 3

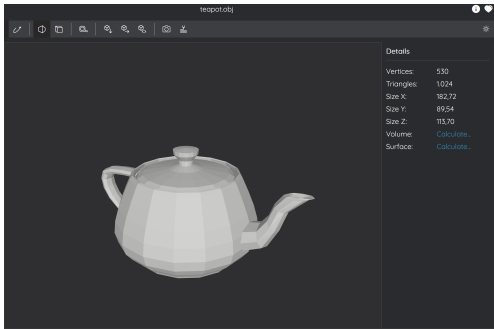
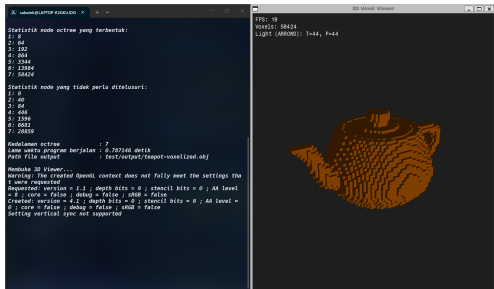
Test Case 4	
Input	Output
 $maxDepth = 7$	

Fig. 4. Hasil Uji Coba Test Case 4

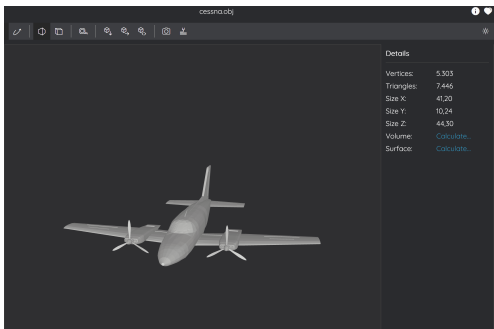
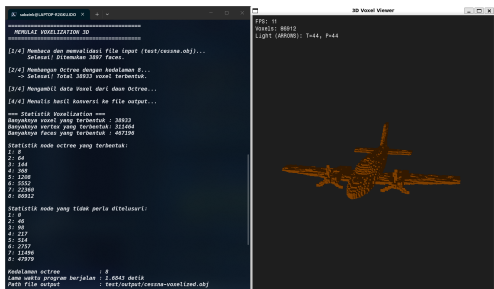
Test Case 5	
Input	Output
 $maxDepth = 8$	

Fig. 5. Hasil Uji Coba Test Case 5

V. PRANALA REPOSITORY

Seluruh kode sumber program dan *test case* dapat diakses secara penuh melalui *repository* GitHub pada tautan berikut:

https://github.com/Rizelbit/Tucil2_13524001_13524056

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Samuelson D. Tanuraharja
13524001



Reinhard Alfonzo Hutabarat
13524056

LAMPIRAN

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil di jalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan concurrency		✓
7	[Bonus] Program dapat menampilkan visualisasi .obj	✓	